

Metric collection from OpenTelemetry (Otel) metrics

- Release version: Australia
-
- Updated May 5, 2026
-
- ⌚ 4 minutes to read

The MID WebService metric Collector enables you to collect JSON and protobuf formatted metrics sent from OpenTelemetry (Otel).

Before you begin

Ensure that you have installed the Event Management Connectors (sn_em_connector) plugin on the ServiceNow AI Platform instance and the ServiceNow version is Australia or above.

Ensure that you:

- Deploy and start the MID Server. See [MID Server configuration](#).
- Configure and start the MID Web Server. See [Configure the MID Web Server extension](#).
- To activate metric collection, ensure that the MID Server that retrieves metrics is configured with the Metric Intelligence extension and that the extension is in Started mode. See [Manually configure the Metric Intelligence extension](#).
- Enable the rest listener.

Role required: evt_mgmt_admin

About this task

Protobuf and JSON formatted metrics are sent from Opentelemetry. The MID Server transforms the collected metrics by parsing them using the TransformMetrics_MidOpenTelemetry script include, located at Event Management > Integrations > PushConnectors. In the Push Connectors page, select OpenTelemetry.

The default format of the URL to push event messages from the GCP to the MID Server is:
`http://{MID_Server_IP}:{MID_Web_Server_Port}/api/mid/sa/inbound_metrics?transform=TransformMetrics_MidOpenTelemetry`

Table 1. Variables in the default URL

Variable	Description
----------	-------------

Variable	Description
MID_Server_IP	IP address of the MID Web Server Extension.
MID_Web_Server_Port	Listening port of the MID Web Server Extension.
MID_Web_Server_User	Username for the MID Web Server Extension.
MID_Web_Server_Password	Password of the user of the MID Web Server.

Procedure

1. CONFIGURE RECEIVERS: HOSTMETRICS RECEIVER - SYSTEM METRICS COLLECTION

Add the following hostmetrics receiver under the "receivers:" section to collect system metrics:

```
receivers:
  hostmetrics:
    collection_interval: 10s
    scrapers:
      cpu:
      memory:
      disk:
      filesystem:
      network:
      processes:
      paging:
```

Note:

- Collection_interval set to 10 seconds for frequent metric updates.
- Adjust the interval based on your monitoring requirements (higher values reduce overhead).
- Enable only the scrapers you need to minimize resource usage.

2. CONFIGURE PROCESSORS

TRANSFORM PROCESSOR - REMOVE UNNECESSARY FIELDS

- Add the following transform processor under the "processors:" section to remove unnecessary fields from the payload:

```
transform:
  metric_statements:
    - context: scope
      statements:
        - set(name, "")
        - set(version, "")
```

b. MEMORY LIMITER PROCESSOR - PREVENT MEMORY ISSUES

Add the following memory_limiter processor under the "processors:" section to prevent out-of-memory errors:

```
memory_limiter:
  limit_mib: 5
  spike_limit_mib: 1
```

Note:

- limit_mib set to 5 MB of memory usage
- spike_limit_mib adjusted to 1 MB to handle temporary spikes
- Adjust these values based on your system's available memory

c. FILTER PROCESSOR - SELECT SPECIFIC METRICS

Add the following filter processor under the "processors:" section to include only selected metrics:

```
filter/minimal:
  metrics:
    include:
      match_type: strict
      metric_names:
        # CPU Metrics
        - system.cpu.time
        - system.cpu.load_average.1m
        - system.cpu.load_average.5m
        - system.cpu.load_average.15m
        # Memory Metrics
        - system.memory.usage
        - system.paging.usage
        # Disk Metrics
        - system.filesystem.usage
        - system.disk.io
        - system.disk.io_time
        # Process Metrics
        - system.processes.count
        - system.processes.created
        # Paging Metrics
        - system.paging.operations
        # Network Metrics
        - system.network.io
        - system.network.packets
        - system.network.errors
        - system.network.dropped
        - system.network.connections
        # MySQL Metrics
        - mysql.operations
```

```

- mysql.locks
- mysql.uptime
- mysql.threads
# HTTP Check Metrics
- httpcheck.duration
- httpcheck.status
- httpcheck.error
# Prometheus node_exporter Metrics
- node_cpu_seconds_total
- node_memory_MemTotal_bytes
- node_memory_MemAvailable_bytes
- node_disk_read_bytes_total
- node_disk_written_bytes_total
- node_disk_io_time_seconds_total
- node_network_receive_bytes_total
- node_network_transmit_bytes_total

```

Note:

Histogram metrics are not supported by ServiceNow MID Server.

Even if histogram metrics are sent to the MID Server, the ServiceNow transform script will automatically ignore them. However, as a best practice, it is recommended to filter out histogram metrics at the collector level to optimize network bandwidth and reduce unnecessary data transmission.

To exclude histogram metrics, add the following filter:

```

filter/drop_histograms:
  metrics:
    metric:
      - 'type == METRIC_DATA_TYPE_HISTOGRAM'

```

3. CONFIGURE EXPORTERS

Choose one of the following authentication methods based on your requirements:

a. OPTION A: API KEY AUTHENTICATION

Add the following exporter under the "exporters:" section:

```

otlphttp/opentelemetry:
  metrics_endpoint: "http://{mid_server_ip}:{port}/api/mid/sa/inbound
  headers:
    Authorization: "Key {api_key}"
  timeout: 30s
  compression: none
  tls:
    insecure: true
  sending_queue:

```

```
enabled: true
num_consumers: 4
queue_size: 100
retry_on_failure:
  enabled: true
  initial_interval: 5s
  max_interval: 30s
```

Note:

Compression: gzip is not supported. Replace {mid_server_ip}, {port}, and {api_key} with your actual values.

b. OPTION B: BASIC AUTHENTICATION

For Basic Authentication, you need to configure both an extension and an exporter.

i. Configure Basic Auth Extension

Add the following under the "extensions:" section:

```
extensions:
  basicauth/mid:
    client_auth:
      username: "otel"
      password: "Otel@123"
```

Note:

Replace username and password with your actual credentials.

ii. Configure Basic Auth Exporter

Add the following under the "exporters:" section:

```
otlphttp/opentelemetryBasic:
  metrics_endpoint: "http://{mid_server_ip}:{port}/api/mid/sa/in
  auth:
    authenticator: basicauth/mid
  timeout: 30s
  compression: none
  tls:
    insecure: true
  sending_queue:
    enabled: true
    num_consumers: 4
    queue_size: 100
  retry_on_failure:
    enabled: true
    initial_interval: 5s
    max_interval: 30s
```

Note:

Compression:gzip is not supported. Replace {mid_server_ip} and {port} with your actual values.

4. CONFIGURE SERVICE PIPELINE

Add the configured processors and exporters to the metrics pipeline under the "service:" section:

```
service:
  pipelines:
    metrics:
      receivers: [<your_configured_receivers>]
      processors: [memory_limiter, transform, filter/minimal]
      exporters: [otlphttp/opentelemetry, otlphttp/opentelemetryBasic]
```

Note:

- Replace <your_configured_receivers> with your actual receiver names (e.g., hostmetrics).
- Include only the exporters you have configured (API Key or Basic Auth, or both).
- Processor order matters: memory_limiter should be first, followed by transform and filter.
- Ensure the processor names match those defined in Step 2.
- If using histogram filtering, add filter/drop_histograms after filter/minimal.

5. ADDITIONAL CONFIGURATION OPTIONS

By default, OpenTelemetry sends data in Protocol Buffers (protobuf) format.

To send data in JSON format instead, add the following header to your exporter configuration:

```
headers:
  Content-Type: "application/json"
```

COMPLETE CONFIGURATION EXAMPLE.

Below is a complete example configuration file combining all components:

```
receivers:
  hostmetrics:
    collection_interval: 10s
  scrapers:
    cpu:
    memory:
    disk:
    filesystem:
    network:
    processes:
    paging:
```

```
processors:
  memory_limiter:
    limit_mib: 5
    spike_limit_mib: 1

  transform:
    metric_statements:
      - context: scope
      statements:
        - set(name, "")
        - set(version, "")

  filter/minimal:
    metrics:
      include:
        match_type: strict
      metric_names:
        - system.cpu.time
        - system.cpu.load_average.1m
        - system.memory.usage
        - system.network.io

  filter/drop_histograms:
    metrics:
      include:
        metric:
          - 'type != METRIC_DATA_TYPE_HISTOGRAM'

extensions:
  basicauth/mid:
    client_auth:
      username: "otel"
      password: "Otel@123"

exporters:
  otlphttp/opentelemetry:
    metrics_endpoint: "https://10.0.0.100:8443/api/mid/sa/inbound_metrics?"
    headers:
      Authorization: "Key your_api_key_here"
    timeout: 30s
    compression: none
    tls:
      insecure: false
    sending_queue:
      enabled: true
      num_consumers: 4
      queue_size: 100
    retry_on_failure:
```

```
enabled: true
initial_interval: 5s
max_interval: 30s
```

```
service:
```

```
extensions: [basicauth/mid]
```

```
pipelines:
```

```
metrics:
```

```
receivers: [hostmetrics]
```

```
processors: [memory_limiter, transform, filter/minimal, filter/drop_
```

```
exporters: [otlphttp/opentelemetry]
```